



GATEWAY TECHNICAL

Neural Networks



UNIVERSITY OF FLORIDA



0. Overview

Deep learning extends classical machine learning to highly complex patterns

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Classical ML (e.g., regression, trees) captures limited, linear, or structured patterns.
 - **Deep learning** uses stacked layers of transformations to learn complex, nonlinear relationships.
 - These layers allow the model to extract **hierarchical features**, simple patterns combine to form complex ones.

This module focuses on vision and generative systems

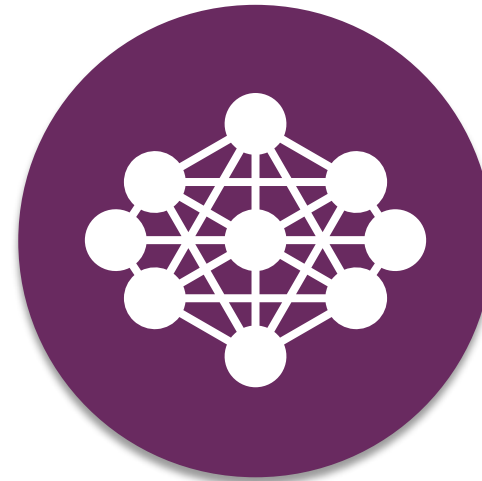
This module focuses on **visual and general deep architectures**, building intuition for:



Neural networks (dense feedforward structures)



Convolutional networks (CNNs) for image data



Autoencoders for compression and anomaly detection



GANs and Diffusion Models for generative synthesis

The deep learning workflow combines data, architecture, and training



A deep learning model is defined by three components:

- 1 Data:** Inputs (e.g., images, text, tabular) that feed into the network
- 2 Architecture:** Layer structure that defines information flow
- 3 Training:** Optimization process that tunes weights to minimize loss

Deep learning thrives when scale and representation matter

- Performance improves as **data size** and **model capacity** increase.
- Large datasets allow networks to learn increasingly abstract and generalizable features.
- However, greater capacity introduces **risks of overfitting** and **training instability**.





1. Introduction to Neural Networks

Prelude: Neural Networks in a Nutshell

Neural networks evolved from **three intertwined ideas**:



Inspiration from biology:
Neurons as connected
signal units



Formalization in mathematics:
Inputs, weights, activations



**Abstraction in
learning theory:**
Approximating any function
from data

In short

It's brain-like → it's math → it's a general function approximator.

Neural networks borrow metaphors from the brain - loosely

- Early researchers drew inspiration from how **neurons fire and connect**.
- Each artificial neuron mimics the idea of **signal activation and propagation**.
- This biological metaphor helped spark the field but is **only a starting analogy**.
- The human brain remains vastly more complex and parallel than any neural net.

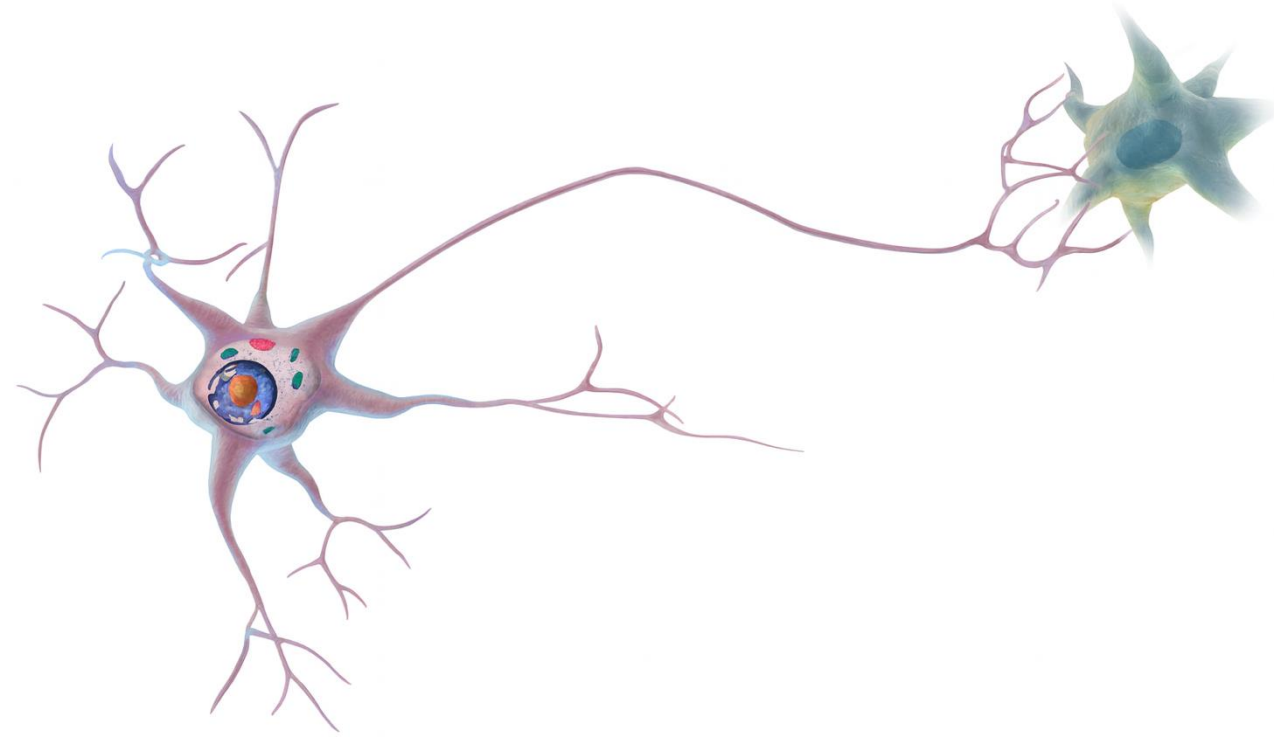


Figure: Illustration of a biological neuron

The mathematical neuron abstracts signals into numbers

- Each “neuron” computes a **weighted sum of its inputs** and applies an activation function.
- Inputs x_i and weights w_i capture learned influence; bias b shifts the response.

The output

$$a = f\left(\sum_i w_i x_i + b\right)$$

Networks are layers of these transformations stacked together.

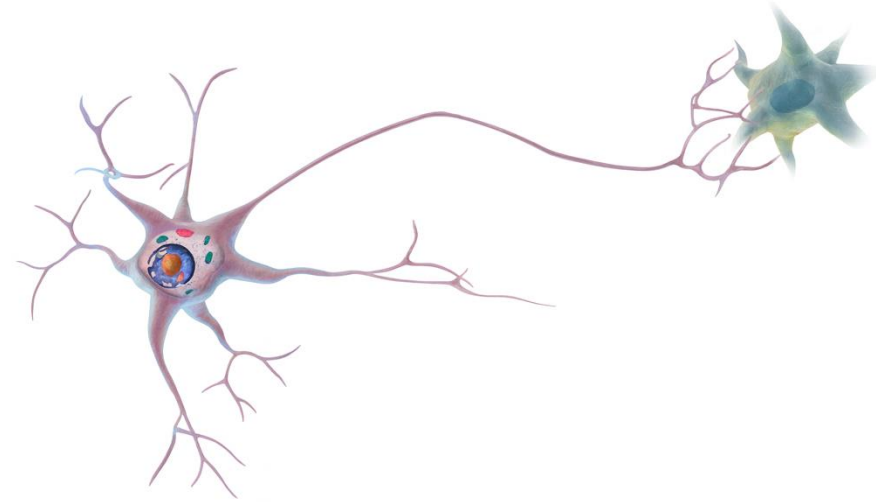


Figure: Diagram of a single artificial neuron

Neural networks are mathematical function approximators

- 
- The goal is to learn an **approximation of a function** that maps inputs $\rightarrow$ outputs.
 - Layers of nonlinear transformations let the model represent **complex, nonlinear patterns**.
 - Neural networks **don't think**. They compute transformations that best fit data.
 - Their power comes from **composition**, not human-like cognition.

Deep networks stack multiple layers to learn hierarchical representations



The input layer receives raw data as numeric features

- The **input layer** is where data enters the network.
- Each neuron in this layer corresponds to a **feature** (e.g., pixel intensity, numeric variable, word embedding).
- Input values are represented as a **vector** $x = [x_1, x_2, \dots, x_n]$.

Figure: Diagram showing feature vector $x_1, x_2, \dots, x_n$ entering the input layer of a neural network

Hidden layers transform inputs into learned representations

- **Hidden layers** apply weights and activations to re-express the data.

Each layer computes

$$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$$

where $a^{(l)}$ are activations and $W^{(l)}$ the learned weights

- These transformations **extract patterns** and **combine features** from earlier layers.

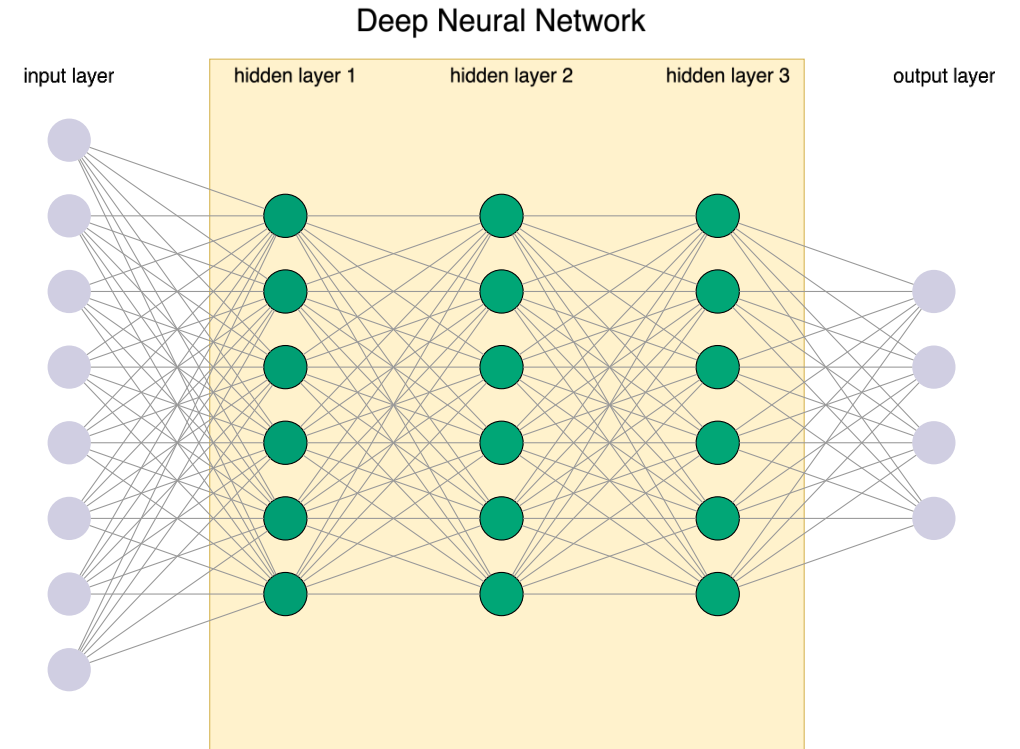


Figure: Illustration of a multi-layer network showing forward flow through hidden layers, with equations beside each transformation

The output layer produces final predictions

The **output layer** maps the final hidden representation to a task-specific output.

- **Regression:** A single neuron producing a continuous value (e.g., price, score).
- **Classification:** One neuron per class, usually with **softmax** to produce probabilities.

The network's prediction is

$$\hat{y} = f_{\text{out}}(W^{(L)}a^{(L-1)} + b^{(L)})$$

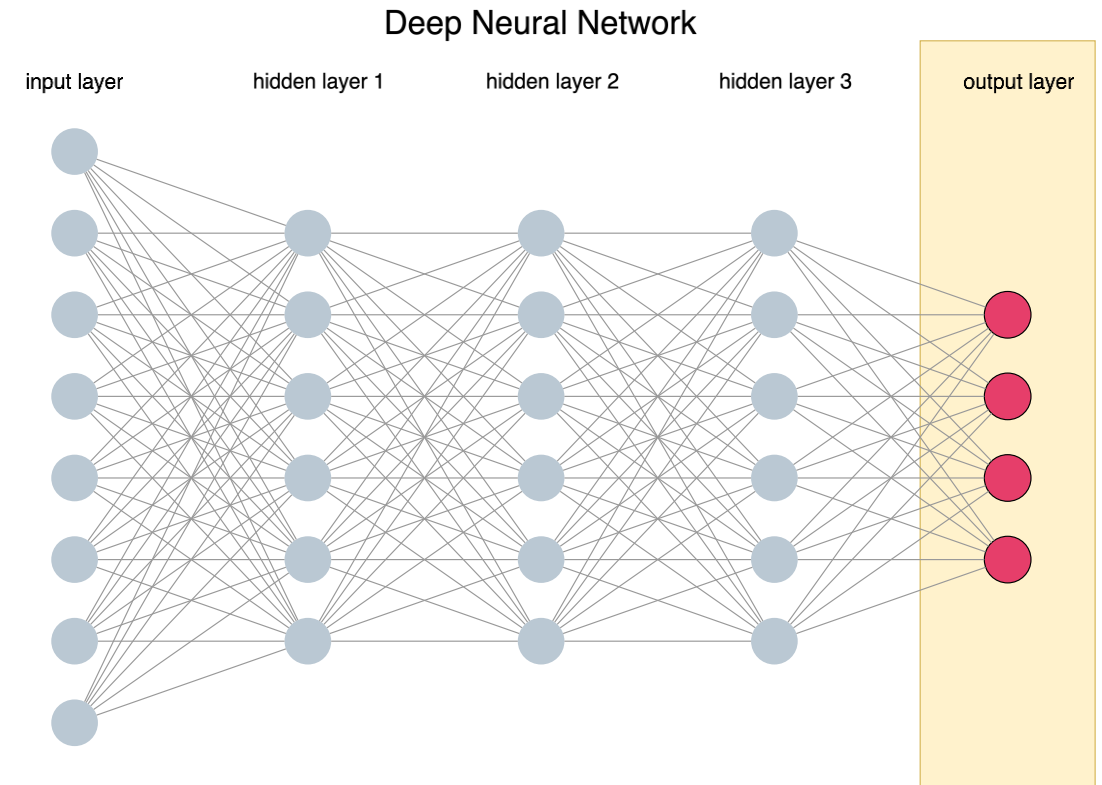


Figure: Diagram highlighting the final output layer with examples — one continuous neuron for regression and multiple softmax neurons for classification

Stacking layers creates hierarchical representations

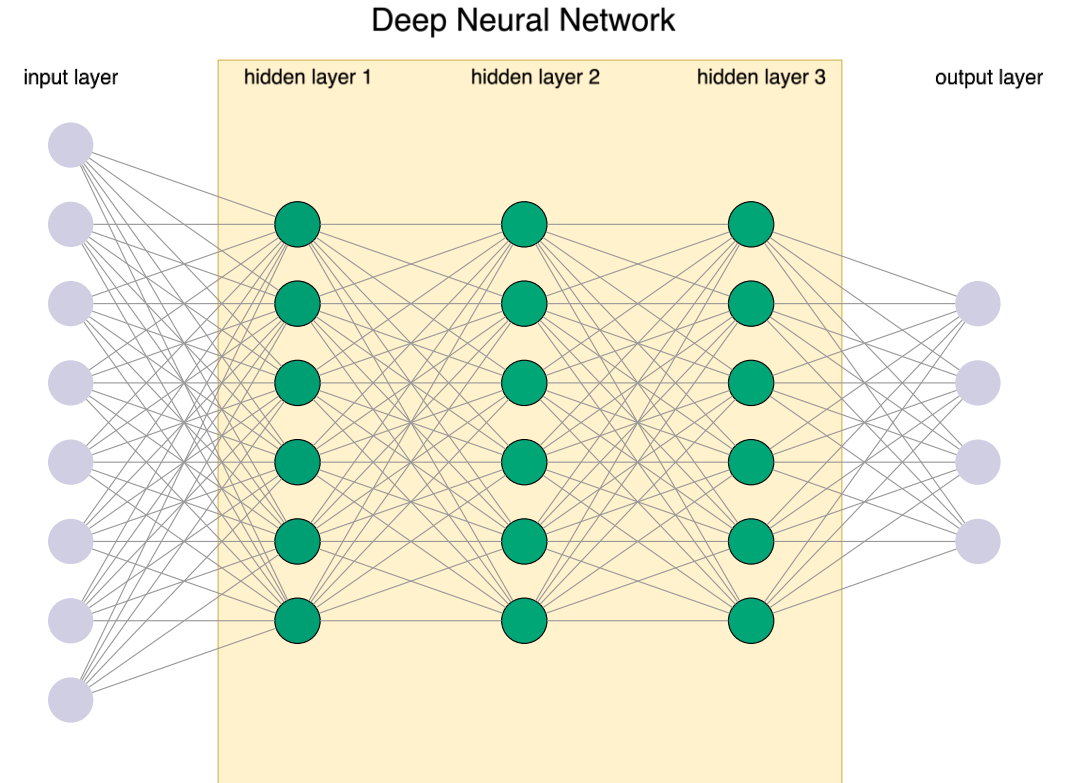
- Each layer learns features based on the **outputs of the previous one**.
- Depth allows the network to learn **hierarchical abstractions**:

Vision

Pixels → edges → shapes → objects

Text

Characters → words → phrases → meaning



Nonlinear activations give neural networks expressive power

- Without nonlinear activations, even deep networks behave like a **single linear model**.
- Nonlinearity lets each layer **warp the feature space**, enabling complex decision boundaries.
- Activations are what make neural networks **universal function approximators**.

Figure: Plot comparing Sigmoid, Tanh, ReLU, and Leaky ReLU

A single neuron combines inputs into an activated output

- Each neuron computes a **weighted sum** of its inputs and applies an **activation**.

Computation

$$z = \sum_i w_i x_i + b \quad \text{and} \quad a = f(z)$$

w_i : learned weights; b : bias term;

- This transforms numeric inputs into a new representation (activation).

Figure: One artificial neuron showing inputs x_i , weights w_i , bias b , summation node, activation function f , and resulting output a .

The forward pass propagates activations layer by layer

In the forward pass, each layer receives activations from the previous layer and computes new ones

$$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$$

- Early layers capture **low-level features**; deeper layers learn **abstract patterns**.
- The network's power comes from **composing many simple transformations**.
- “Deep learning” = learning through **hierarchical composition**.

Learning starts by defining a loss that measures error

- A **loss function** quantifies how far predictions are from true outcomes.
- The network's goal is to **minimize this loss** across all training examples.

Common examples

- **Regression:** Mean Squared Error (MSE)

$$L = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$

- **Classification:** Cross-Entropy Loss

$$L = - \sum_i y_i \log(\hat{y}_i)$$

- The loss turns *performance* into a **number the model can optimize**.

Backpropagation uses gradients to update the network

- **Backpropagation** distributes the loss backward through the network.
- It computes how each weight contributes to the total error (**credit assignment**).

Each weight w is updated using its gradient

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

- This repeats for all layers ...
collectively this is called **gradient descent**.

The **learning rate** (η) sets the step size in gradient descent



Model depth changes the balance between bias and variance



Bias	Error from overly simplistic assumptions → underfitting
Variance	Sensitivity to noise → overfitting
Shallow networks	High bias, low variance → simple but limited capacity
Deep networks	Low bias, high variance → powerful but risk memorization

Figure: Classic bias–variance curve showing test error as a U-shape, with annotated points for underfitting (high bias), optimal capacity, and overfitting (high variance)

Regularization adds guardrails, reining variance in, so deep nets generalize



Figure: A network without regularization (left) and one with (right).
Network weights are reduced to reduce the impact of high weight inputs

Putting it together: Pick the right guardrails for your setting



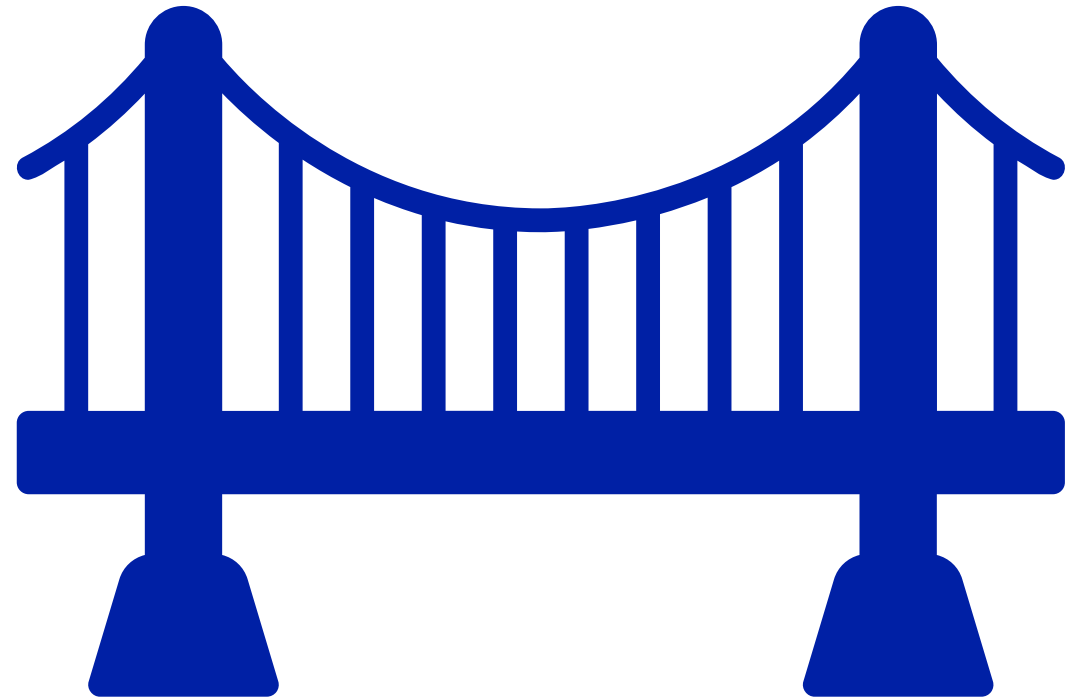
Limited data	Strong augmentation, transfer learning (freeze), smaller head, early stop
Plenty of data, unstable training	Weight decay + dropout; consider LR schedule; batch/seed discipline
Overfit signs (train↑/val↓)	Increase regularization/augmentation; reduce capacity
Underfit signs (both flat/high)	Ease regularization; add capacity; adjust learning rate



2. Practical Training Foundations

Bridge: From what networks are to how we train them

- We've covered **what** neural networks are: units, layers, activations, loss, gradients.
- Next: **How** we make them train well in practice.
- We'll mainly focus on **optimizers, initialization, normalization, and training choices**



Optimizers control how networks navigate the loss landscape



Stochastic Gradient Descent (SGD)

Updates weights using small random batches

Momentum

Adds velocity to smooth noisy updates

Adam

Adaptive learning rates per parameter; fast convergence

Choice of optimizer affects **speed** and **stability** of training

Normalization keeps activations stable during training

Normalization layers

Normalization layers rescale activations to control magnitude drift between layers

Batch Normalization (BatchNorm)

Normalizes within each mini-batch; helps deep nets train faster

Layer Normalization (LayerNorm)

Normalizes across features; works better for sequential or transformer models

Benefits

- Smoother gradients
- Higher learning rates possible
- Less sensitivity to initialization

Prelude: How fast we move → How long we train

Training behavior is shaped by two kinds of choices:

1

How fast we move:

The optimization dynamics
(learning rate, batch size.)

2

How long we train:

The training schedule
(epochs, early stopping.)

- **These interact:** Fast updates need careful stopping; slower learning may need more passes.
- Finding balance between speed and duration drives both **stability** and **generalization**.

Learning rate and batch size define optimization behavior

Learning rate (η)

(Recap) controls the step size of each weight update

- Too high $\rightarrow$ oscillation or divergence
- Too low $\rightarrow$ slow convergence

Batch size

- **Large batches** $\rightarrow$ smoother gradients but may overfit or get stuck in sharp minima
- **Small batches** $\rightarrow$ noisier updates, often better generalization

Together, they shape how the model **explores** the loss landscape.

Epochs and early stopping control training duration



Epochs

- One full pass through the training data.
- More epochs $\neq$ always better — risk of overfitting after a point.

Epochs and early stopping control training duration

- **Early stopping:** Monitor validation loss and stop when it stops improving.
- Saves compute and prevents memorization of training noise.
- Combine with learning curves to find the **sweet spot for generalization**.

Figure: Training and validation loss curves over epochs — overfitting occurs after a certain point.

Train, validation, and test sets serve different purposes

Split data to measure model performance fairly

- **Training set:** Fit model parameters
 - **Validation set:** Tune hyperparameters and detect overfitting
 - **Test set:** Final, unbiased evaluation of generalization
- Typical proportions: **70/15/15** or **80/10/10**, depending on data size.
 - Keep validation and test sets **untouched during training**.

Good data hygiene underpins trustworthy evaluation



Maintain class balance

Use stratified sampling to mirror real distributions.

Prevent data leakage

Ensure test data contains no information seen during training.

Ensure reproducibility

Record random seeds and preprocessing steps.

Document data provenance

Keep clear records of data sources and transformations.

These practices make results **replicable and defensible**.



3. Implementing Feedforward Neural Networks

Neural networks can be built in a few tools- here we'll use keras.

- Keras provides a **high-level interface** for defining deep learning models.
- The **Sequential API** allows for a simple layer-by-layer stack (e.g., Input → Dense(ReLU) → Dense(ReLU) → Output(Sigmoid)).
- Focus more on **architecture design** and **parameter choices**, focus less on code (it's easy to go get somewhere).

Each layer automatically manages its weights and biases during training.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(32, activation='relu', input_shape=(n_features,)),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

The training workflow follows the pattern: Compile → fit → evaluate

`compile()`

- Optimizer (e.g., Adam, SGD)
- Loss (e.g., cross-entropy, MSE)
- Metrics (e.g., accuracy, AUC)

`fit()`

`fit()` runs the training loop: specify epochs, batch size, validation data.

`evaluate()`

`evaluate()` measures final performance on held-out data.

Iterate by adjusting architecture or hyperparameters.

Visualizing loss and accuracy can reveal model behavior during training (will show in the demo)

Plot training and validation loss or accuracy per epoch:

- **Healthy learning:** Both decrease smoothly and stabilize.
- **Overfitting:** Training improves while validation worsens.
- **Underfitting:** Both remain high and flat.

Learning curves provide **diagnostic feedback** for tuning learning rate, architecture, or regularization.

Overfitting can be controlled with dropout, weight penalties, and early stopping

Dropout

Randomly removes neurons during training → forces redundancy (no longer “cool”)

L2 regularization (weight decay)

Penalize large weights to prevent memorization

Early stopping

Monitor validation loss and halt training when it stops improving

Combined, these act as **guardrails** for high-capacity models.

Connect to bias–variance tradeoff

Regularization raises bias slightly to reduce variance

Evaluation metrics and calibration - assess the models fit



Classification metrics

- **ROC-AUC** → ability to separate positive vs. negative classes
- **PR-AUC** → precision–recall tradeoff for imbalanced data
- **Confusion matrix** → specific error types

Calibration curves

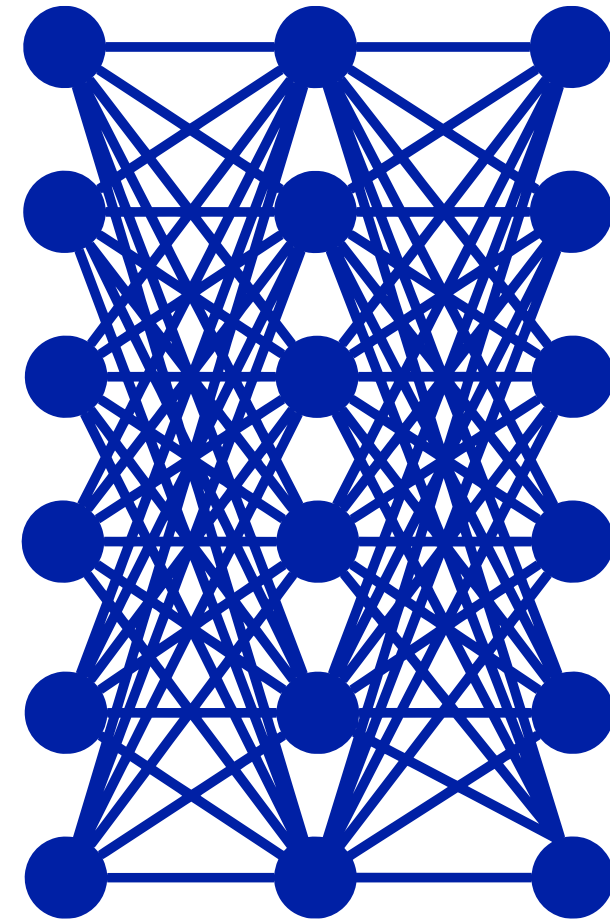
Check whether predicted probabilities match true frequencies.

Threshold tuning

Choose probability cutoffs to maximize business value or F1-score.

Neural networks are not always the right tool for every problem

- Deep networks excel on **large, high-dimensional, nonlinear** problems.
- For **small datasets** or **simple relationships**, simpler models (e.g., logistic regression, decision trees) often perform better.
- **Interpretability, compute cost, and tuning complexity** may outweigh accuracy gains.
- The right question is not “Can I use a neural net?” but “Do I need one?”





4. Convolutional Neural Networks (CNNs)

Why convolution for vision?

- Dense layers treat every pixel position **independently** → miss spatial structure.
- Images have **locality**: Nearby pixels relate; patterns repeat across positions.
- **Convolution** exploits this with small, shared filters that slide over the image.
- **Result**: Fewer parameters, faster learning, and features that **generalize across location**.



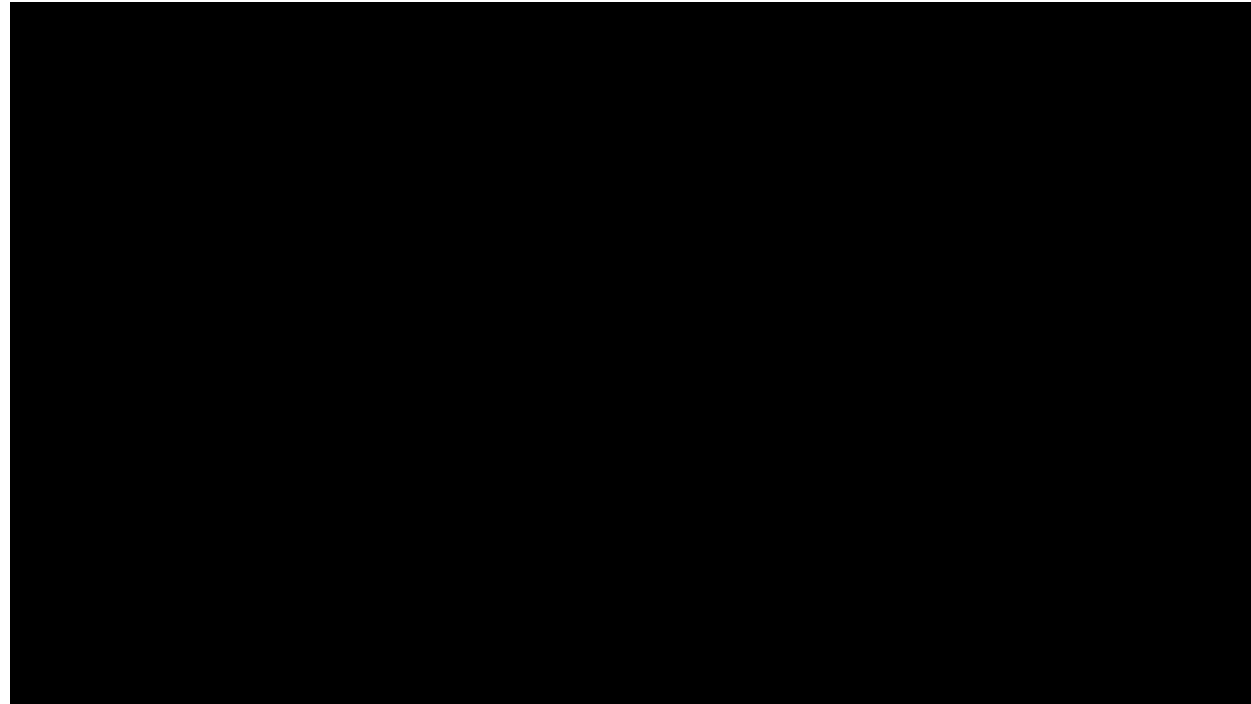
Convolutional layers detect patterns in space rather than across features

- **Convolution** applies a small filter (kernel) across an image to detect local patterns.
- Filters learn to respond to specific visual structures — edges, corners, textures.
- **Feature maps** record where those patterns occur spatially.
- Stacking multiple convolutional layers allows hierarchical feature extraction.



A convolution slides a small filter across an image to extract local patterns

- The **kernel (filter)** moves across the input, computing a weighted sum at each location.
- Each output value = local pattern strength in that region.
- This operation is **translation invariant** — same weights are applied everywhere.
- Convolutions detect edges, corners, or textures depending on learned weights.



Convolution mathematically is local weighted averaging

Different kernels emphasize different patterns (edges, blurs, textures).



Identity

The original



Blur

De-emphasizes differences in adjacent pixels



Sharpen

emphasizes differences in adjacent pixel values.



Sobel

Show only in differences in adjacent pixel values in a direction (this is a bottom sobel).

Stride determines how far the kernel moves each step

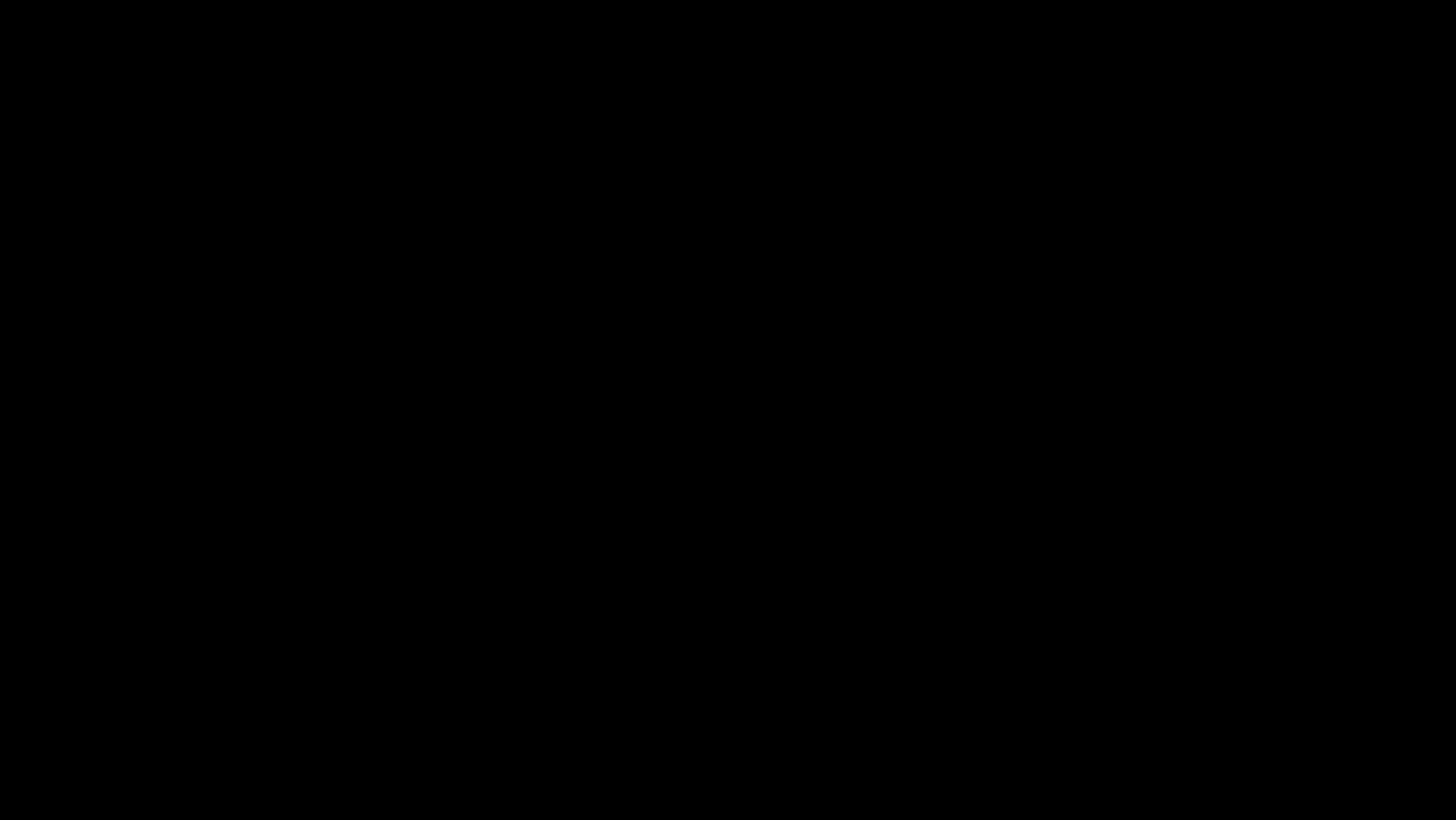


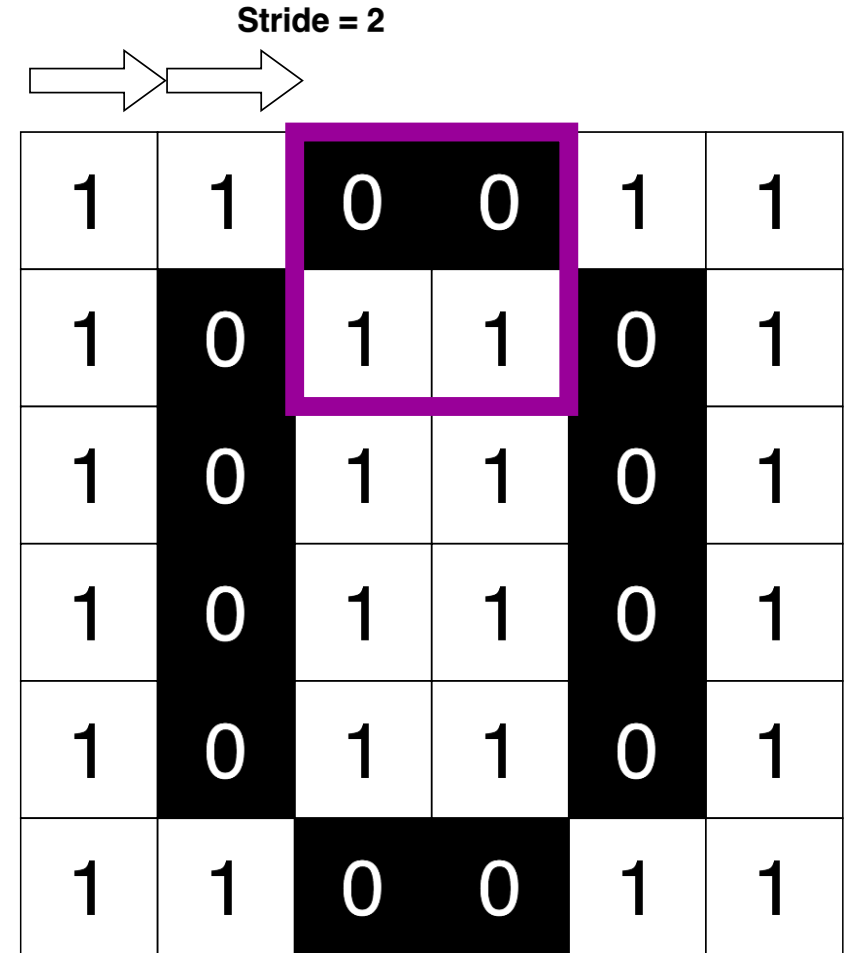
Figure: Diagram showing a 3×3 kernel sliding across an image with stride 1

Stride determines how far the kernel moves each step

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- **Stride** = The number of pixels the convolution filter shifts at a time.
 - **Stride = 1:** Maximum detail — overlapping receptive fields.
 - **Stride = 2:** Halves the spatial resolution — faster but less precise.
 - Larger stride → fewer computations, lower spatial resolution.

Stride determines how far the kernel moves each step

- **Stride** = The number of pixels the convolution filter shifts at a time.
- **Stride = 1**: Maximum detail — overlapping receptive fields.
- **Stride = 2**: Halves the spatial resolution — faster but less precise.
- Larger stride → fewer computations, lower spatial resolution.



Padding preserves spatial dimensions at image boundaries

- **Padding** = Adding extra pixels (often zeros) around the border before convolution.
- Prevents the output from shrinking after each layer.
- Used to preserve spatial resolution in deep networks.

Padding = 1

0	0	0	0	0	0	0	0
0	1	1	0	0	1	1	0
0	1	0	1	1	0	1	0
0	1	0	1	1	0	1	0
0	1	0	1	1	0	1	0
0	1	0	1	1	0	1	0
0	1	1	0	0	1	1	0
0	0	0	0	0	0	0	0

Stride, padding, and kernel size together determine output dimensions

Output feature-map size depends on four factors

I = input size

K = kernel size

P = padding

S = stride

Formula

$$O = \frac{I - K + 2P}{S} + 1$$

Increasing stride reduces O ; increasing padding increases O .

Real images have multiple color channels, not just one

- 
- A color image has **three channels** — Red, Green, and Blue (RGB).
 - Each channel is a 2D grid of pixel intensities.
 - A convolutional filter must process **all channels together**, not independently.
 - Each filter therefore has **depth = number of input channels** (e.g., 3 for RGB).

Multiple filters produce stacked feature maps — a 3D tensor

- A CNN layer doesn't use just one filter, it learns **many**, each capturing a different pattern.
- Each filter produces **one 2D feature map**.
- Stacking these maps forms a **3D output tensor** with dimensions:

$$\text{Output shape} = (H, W, \text{\#filters})$$

- **Example:** 32 filters $\rightarrow$ 32 feature maps $\rightarrow$ output depth = 32

Figure: Many layers and kernels are used together to form the “map”

Convolution layers are followed by nonlinear activations

- After each convolution, we apply an **activation function** (usually ReLU).
- ReLU keeps positive activations, zeroes out negatives → sparsity.
- The sequence **Conv** → **ReLU** → **Pool** defines early CNN blocks.
- Without activation, multiple convolutions collapse into one linear filter.

Pooling summarizes nearby activations while reducing size



Max pooling	Keeps strongest activation within each window
Average pooling	Keeps mean value → smoother output
Benefits	• Adds translational invariance • Reduces computation
Drawback	Loss of spatial precision (small shifts may move maxima)

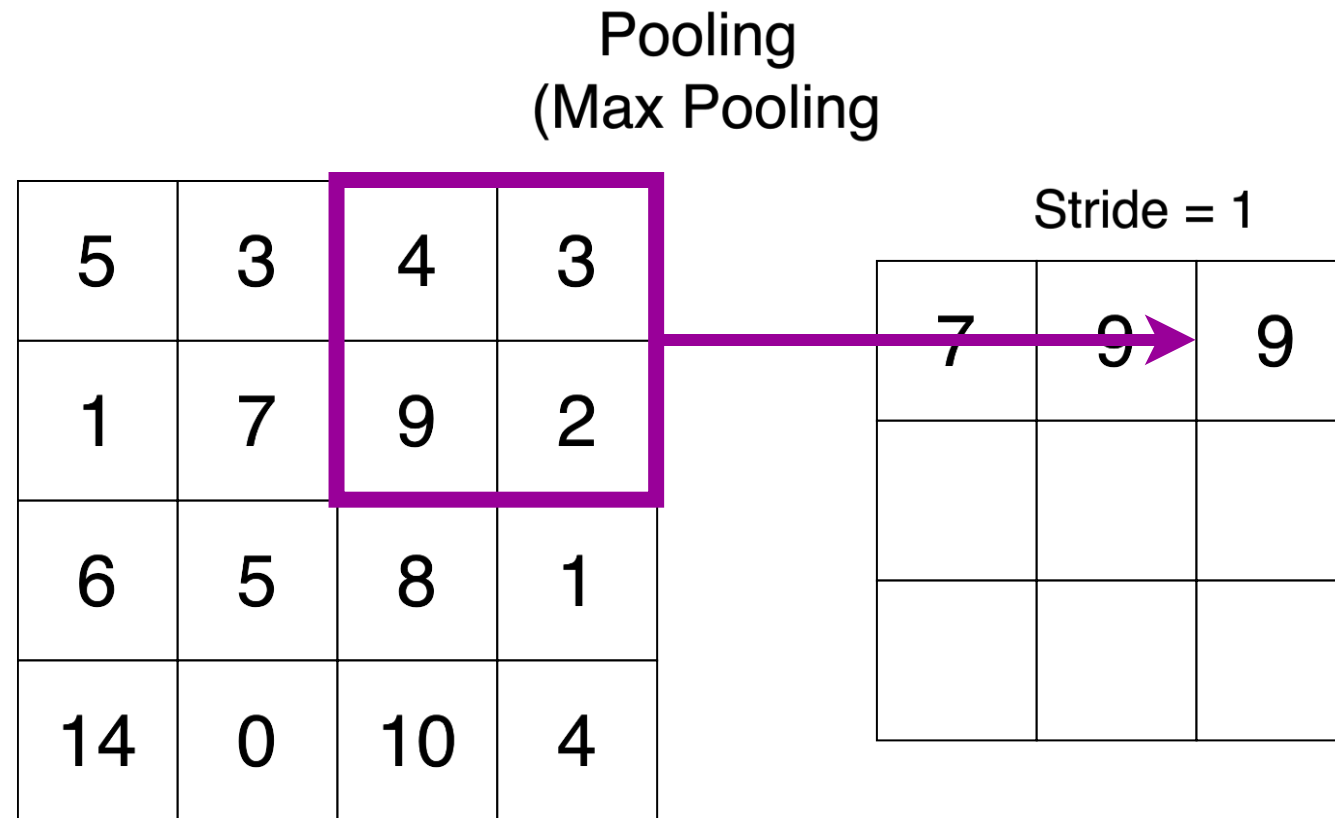
Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1

Pooling
(Max Pooling)

5	3	4	3
1	7	9	2
6	5	8	1
14	0	10	4

Stride = 1

7	9	9
7	9	9



Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1



Example of Max Pooling, Stride 1

Pooling
(Max Pooling)

5	3	4	3
1	7	9	2
6	5	8	1
14	0	10	4

Stride = 1

7	9	9
7	9	9
14	10	10

Example of Max Pooling, Stride 1



When to use pooling vs stride-2 convolutions



Max and avg pool

Simple and fast; can introduce aliasing or lose fine detail.

Stride-2 convolution

Learnable downsampling; preserves discriminative patterns better in convnets.

Heuristic

For classification, GAP + stride-2 convs is a strong default; add blur-pool if inputs shift/alias.

Deeper CNNs build hierarchical feature representations



Data augmentation increases diversity and prevents overfitting

Real-world image datasets are limited.
Augmentation synthetically expands them.

Common techniques

- **Geometric:** Flips, rotations, crops, scaling
- **Color-based:** Brightness, contrast, hue jitter

Occlusion-based

Cutout, Mixup, CutMix (advanced)

Figure: Original image and augmented variants
(flipped, rotated, color-shifted)

Transfer learning allows reuse of pretrained visual knowledge

- Pretrained CNNs (e.g., VGG, ResNet, Inception) learned rich visual features on ImageNet.
- Instead of training from scratch:

1

Freeze the backbone (shared layers).

2

Replace the classification head with a task-specific output layer.

3

Fine-tune upper layers on your dataset if sufficient data.

- Saves compute and improves performance on small datasets.

Fine-tuning decisions depend on data size and shift

- **Small dataset, small domain shift** → freeze backbone, train head only
- **Moderate dataset or moderate shift** → unfreeze top block(s), use lower LR
- **Large dataset or large shift (e.g., RGB aerial vs. medical)** → unfreeze more blocks, stronger regularization
- Watch for **catastrophic forgetting**: Source-domain performance collapses during tuning



Interpretability tools reveal what CNNs “see” inside the network

- CNNs are powerful but opaque. Interpretability methods make internal patterns visible (sort of).
- **Common techniques:**

Saliency maps

Gradient-based sensitivity to input pixels

Grad-CAM

Highlights image regions influencing the prediction

Occlusion sensitivity

Measures output change when masking regions

These tools provide qualitative assurance of model reasoning.

Demonstration: CNNs excel at real-world image recognition tasks

- 
- **Example dataset:** *cars vs. boats or parking lot occupancy.*
 - Compare dense NN vs. CNN performance on same dataset.
 - CNN achieves higher accuracy and faster convergence.
 - Visualize learned **feature maps** and **activations** at different depths.

When training misbehaves: A quick troubleshooting checklist



Loss explodes early?

Lower LR; add warmup; check data scaling; clamp gradients (clip norm/val); verify labels.

Loss plateaus?

Slightly increase LR; add BatchNorm/LayerNorm; try 1–2 extra layers; check for vanishing grads.

Overfitting?

Increase augmentation; add dropout/weight decay; use smaller head (GAP); early stop; collect more data.

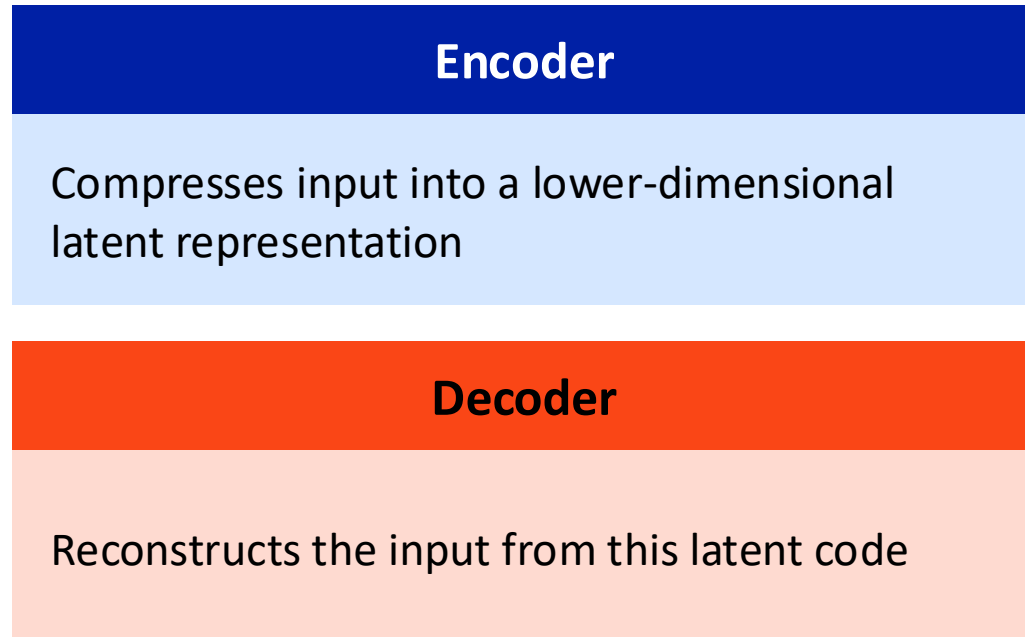
Validation swings?

Use larger/stratified val set; compute **PR-AUC**; smooth curves; average last N checkpoints.



5. Autoencoders

Autoencoders learn to compress and reconstruct data through two linked networks



The encoder–decoder objective teaches compact feature representations

The network minimizes

$$L = \| x - \hat{x} \|^2$$

where x = input, $\hat{x}$ = reconstruction

- **Encoder** extracts meaningful patterns → latent vector z .
- **Decoder** uses z to reconstruct the original input.
- Bottleneck forces the model to **learn what matters most** about the data.

Autoencoders provide flexible tools for dimensionality reduction and denoising



Dimensionality reduction

Nonlinear alternative to PCA — captures complex relationships

Denoising autoencoders

Learn to reconstruct clean inputs from noisy versions

- Add random noise to input ($x + \epsilon$) during training
- Model learns to remove noise → builds robustness

Undercomplete autoencoders learn compressed latent features useful for downstream models.

Autoencoders can detect anomalies by reconstruction error

- When trained on **normal data**, the autoencoder learns to reconstruct expected patterns.
- **Anomalies** deviate from these patterns → yield higher reconstruction errors.

Compute reconstruction loss
for each instance:

$$e_i = \|x_i - \hat{x}_i\|^2$$

Higher (e_i) indicates potential outliers.

- **Common applications:** Fraud detection, defective product inspection, medical scans

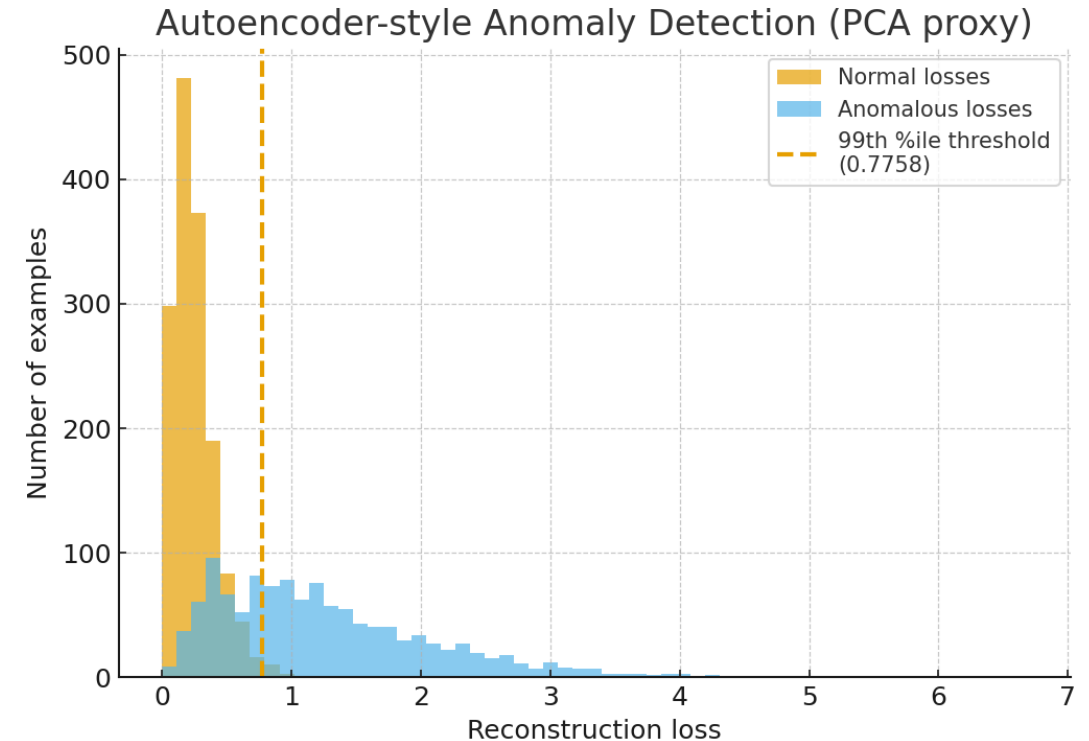


Figure: Reconstruction error distribution. Normal data clustered at low error, anomalies in the high-error tail

Demonstration: autoencoders learn patterns, not labels

Example dataset

credit risk or *churn* with mostly normal cases

Steps

1. Train autoencoder on normal subset.
2. Compute reconstruction errors for all samples.
3. Visualize error distribution — flag anomalies above threshold.

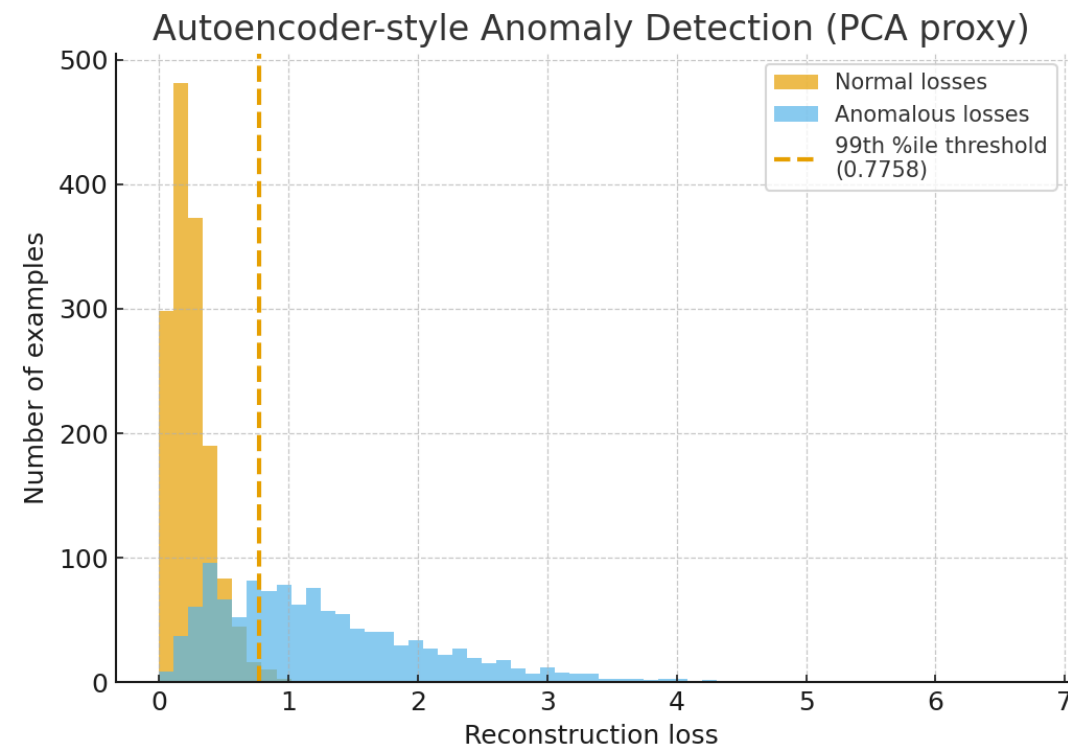
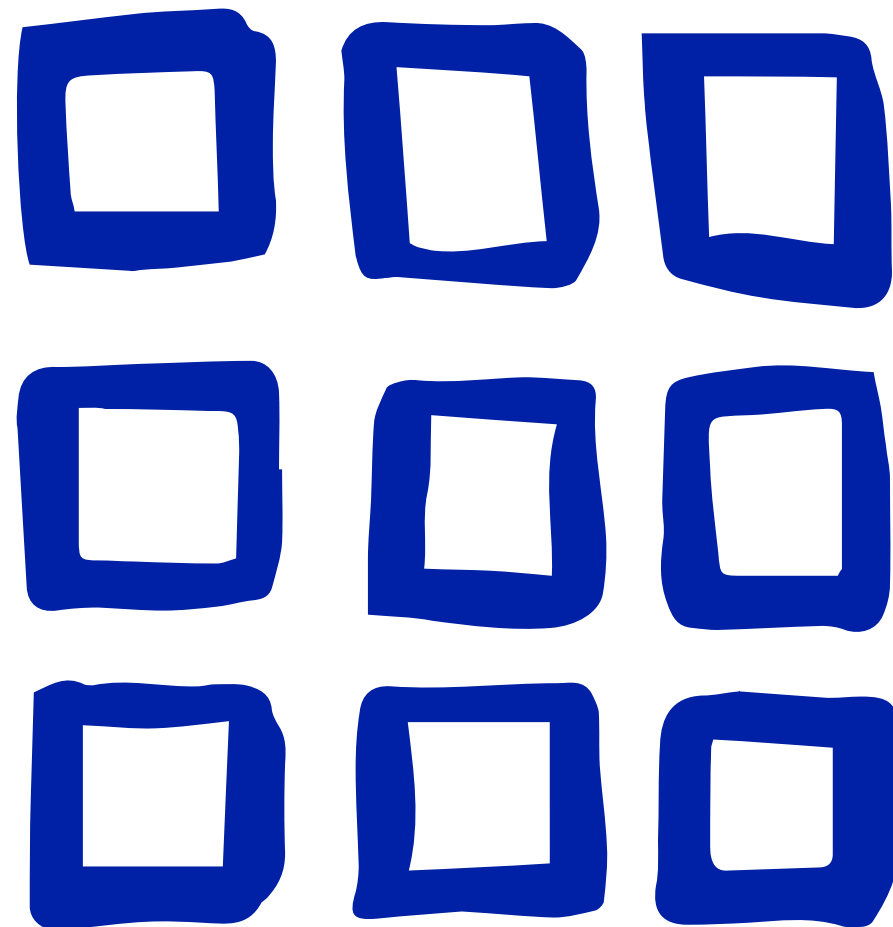


Figure: Reconstruction errors, highlighting threshold and detected anomalies

Autoencoders are early examples of deep representation learning

- Learned latent codes capture **structure** without labels.
- These representations can be reused for clustering or visualization.
- **Variational Autoencoders (VAEs)**: Probabilistic extension adding structure to latent space
- Modern **diffusion models** and **transformers** build upon this concept of learned representations.





6. GANs: Generative Adversarial Networks

GANs pit two networks against each other to create realistic data

- A **Generative Adversarial Network (GAN)** has two parts:

Generator (G)	Discriminator (D)
Creates synthetic samples from random noise	Distinguishes real data from generated data

- Both models train **simultaneously and competitively**:

G improves at <i>fooling</i> D.	D improves at <i>detecting</i> fakes.
---------------------------------	---------------------------------------

- The system reaches an equilibrium when fake and real data are indistinguishable.

Figure: Generator create fake sample from noise → discriminator classifying real vs. fake

GANs can generate strikingly realistic examples of data

- 
- Once trained, the **generator** can create entirely new, realistic samples:
 - Human faces
(e.g., “thispersondoesnotexist.com”)
 - Handwritten digits or objects
 - Synthetic business data
(e.g., fake transactions for simulation)
 - The output quality depends on architecture depth, dataset size, and training stability.

GAN training is unstable and prone to failure modes

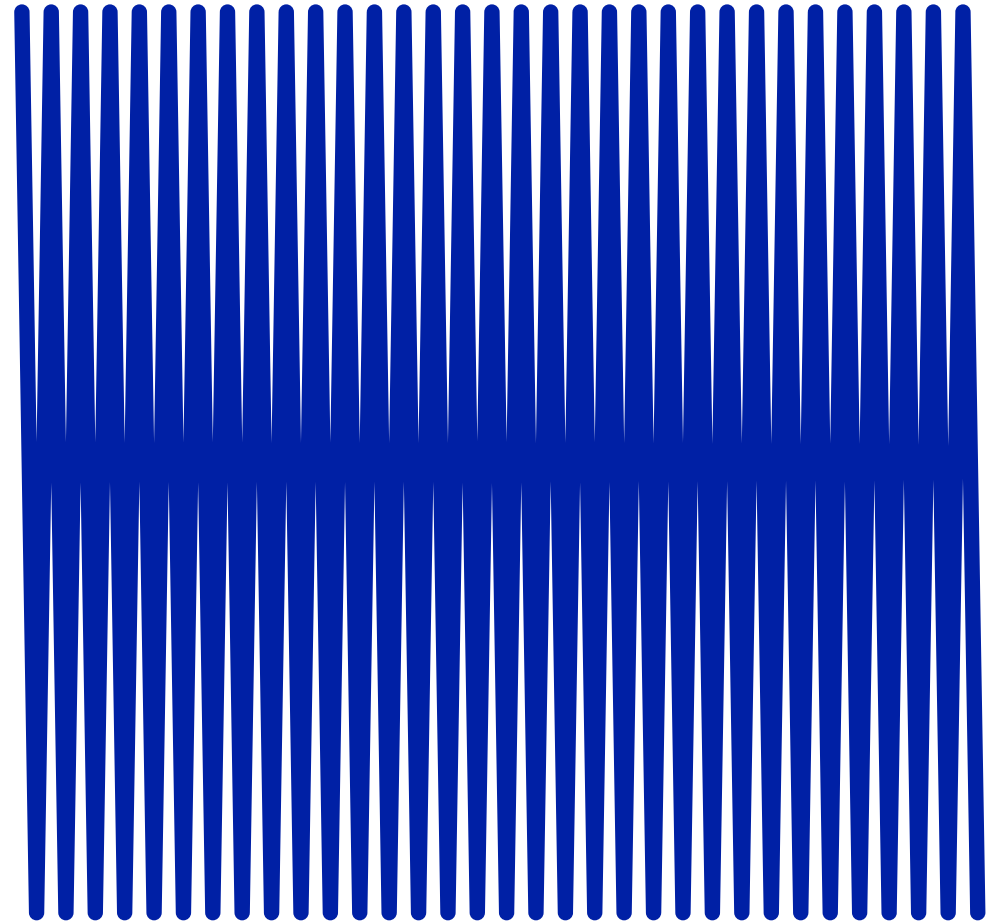
Mode collapse	Generator produces limited variety (e.g., same faces repeatedly)
Oscillating training	Generator and discriminator never converge
Vanishing gradients	Discriminator becomes too strong → generator stops learning
Hyperparameter sensitivity	Learning rates and batch sizes greatly affect outcomes
Improvements include	• Wasserstein GAN (WGAN): Uses continuous distance metric for smoother gradients • Gradient penalty (WGAN-GP): Stabilizes optimization further



7. Diffusion Models (Conceptual Overview)

Diffusion models generate data by reversing a gradual noising process

- 
- **Core idea:** Teach a neural network to **reverse the process of adding noise** to data.
 - The model starts with random noise and learns to reconstruct structured samples step-by-step.
 - Generation = *progressive denoising* — turning randomness into order.



Diffusion models generate data by reversing a gradual noising process



The forward and reverse diffusion processes define the model's learning objective

Forward diffusion

- Gradually adds Gaussian noise to an image or data sample over (T) steps.
- After enough steps, the data becomes indistinguishable from pure noise.

Reverse denoising

- A neural network learns to remove small amounts of noise at each step.
- Starting from noise, the network reverses diffusion to recreate new samples.

Each step refines structure slightly → final image emerges after thousands of iterations.

Diffusion models combine the best traits of GANs and VAEs

Model	Strength	Weakness
GAN	Sharp, realistic outputs	Unstable, mode collapse
VAE	Stable training, probabilistic latent space	Blurry outputs
Diffusion	Stable <i>and</i> realistic	Computationally slow at inference

Diffusion overtook GANs because it offers stability, diversity, and control



Training stability	Simple, likelihood-based objective avoids adversarial instability.
Output diversity	Each sampling path can generate unique variations.
Controllability	Text conditioning, guidance scales, and latent diffusion enable fine control.

- Today's generative AI systems (e.g., Stable Diffusion, Imagen, Sora) build directly on this paradigm.
- Diffusion is the **new foundation** for high-quality, controllable generation.



Hands-On Exercise: Capstone

The capstone reinforces conceptual understanding through practical modeling



Session

This session integrates all major network types from the module.

We will

- Build and train small neural networks on structured and image data.
- Compare model types for suitability, interpretability, and performance.

Emphasis

Learning through experimentation. We will tackle several tasks that highlight a key deep learning concept

Part 1: Train a Feedforward Neural Network for tabular prediction

Dataset options

`credit_risk.csv` or
`synthetic_blobs.csv`

Task

Predict a binary label
(e.g., loan approval, class membership)

Steps

1

Build and train a dense neural network (2 hidden layers).

2

Compare against a logistic regression baseline (Module 5).

3

Evaluate metrics: Accuracy, ROC-AUC, calibration.

Reflection

Does the neural network offer meaningful improvement?

Part 2: Use an Autoencoder to detect anomalies in tabular data

Dataset

creditcard.csv or churn data with rare events

Steps

- 1 Train autoencoder on the **majority (normal)** class.
- 2 Compute **reconstruction errors** for all samples.
- 3 Select threshold to flag anomalies (e.g., top 2% of errors).
- 4 Compare to Isolation Forest from Module 4.

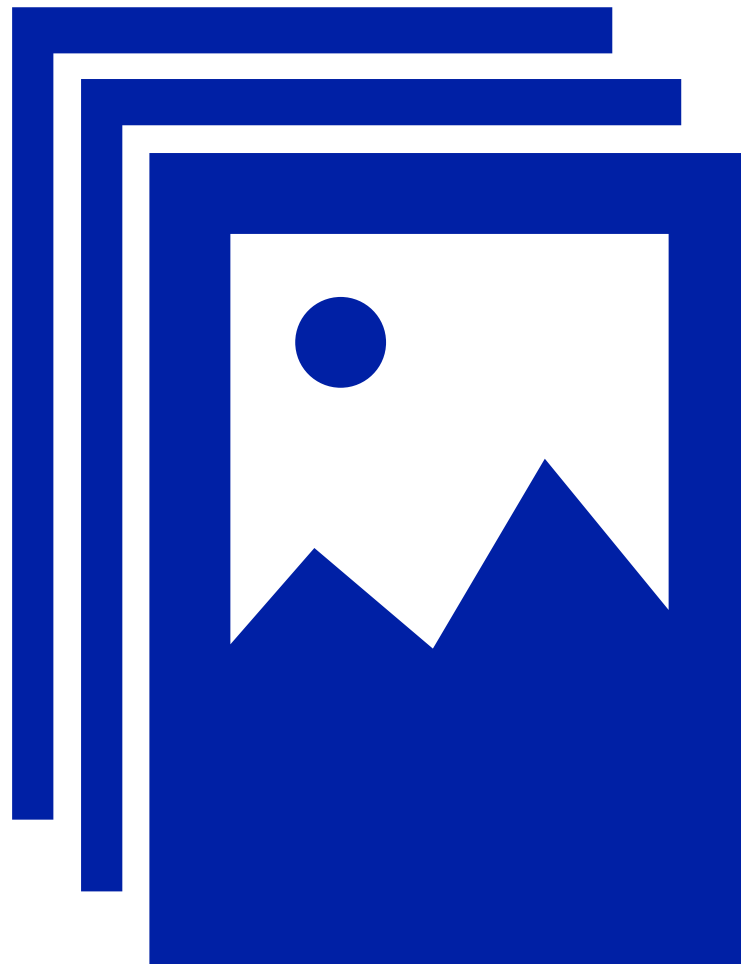
Evaluate detection quality with confusion matrix and F1-score.

Part 3: Build a simple CNN for visual classification

Dataset		Task	
<i>cars vs. boats or parking lot occupancy</i>		Classify image category or occupancy status	
Steps	1	Construct lightweight CNN (1–2 convolutional + pooling layers).	
	2	Train and evaluate on small dataset.	
	3	Visualize learned filters and feature maps .	
	4	Compare to performance of a dense network trained on the same images.	
Observation		CNN should learn faster and generalize better.	

Part 4: Explore pretrained GAN outputs (optional extension)

- Display pre-generated examples from a pretrained GAN (faces, digits, or synthetic business data).
- Discuss visible differences between real and fake outputs.
- Optionally adjust generator noise inputs to see variation in outputs.
- **Debate:** “How do we define realism in generative models?”
- Highlight artifacts (e.g., asymmetric faces, texture errors) and discuss why they occur.



Reflect on which architectures are best suited to different problem types



Feedforward NNs	Flexible baselines for structured/tabular data
CNNs	Ideal for spatially correlated inputs (images, grids)
Autoencoders	Uncover structure and detect anomalies
GANs (and diffusion)	Generate or simulate new data
Key lesson	Match the model to the data and task , not the other way around